
Algorithmique 3 : structures de données arborescentes

Fiche de TP n° 0

Travaux pratiques : modalités et réglages

Environnement

Se reporter à la section de même nom du support d'*Algorithmique 2*.

Modalités

Se reporter à la section de même nom du support d'*Algorithmique 2*.

Réglages de l'EDI Geany

Se reporter à la section de même nom du support d'*Algorithmique 2*.

Greffons Geany intéressants

Se reporter à la section de même nom du support d'*Algorithmique 2*.

Raccourcis de l'EDI Geany

Se reporter à la section de même nom du support d'*Algorithmique 2*.

Suite des réglages

Se reporter à la section de même nom du support d'*Algorithmique 2*.

Make sous Geany

Se reporter à la section de même nom du support d'*Algorithmique 2*.

Algorithmique 3 : structures de données arborescentes

Fiche de TP n° 1

Utilisation du TDA table et du module fourretout

Objectifs : utilisation d'un module polymorphe implantant le TDA table par la méthode de hachage avec chaînage séparé et du module « fourretout ».

Prérequis : chapitre du cours sur le TDA table.

Travail minimum : exercices 1 à 3.

Commencez par récupérer le dossier 1/, à trouver dans le dossier `algo3_src/tp/`.

Les sous-dossiers `hashtable/`, `hashtable_test/` et `holdall/` du dossier sont des copies des sous-dossiers de mêmes noms du dossier `algo3_src/courstd/`.

C'est dans le sous-dossier `hashtable_test/` que vous travaillerez les exercices 1 à 4. L'exercice 5 est essentiellement à travailler dans le sous-dossier `holdall/`.

Dans la version originelle du source `hashtable_test/main.c`, un compteur d'occurrences est alloué pour chacun des mots différents lus sur l'entrée.

Vous devez envisager ici une autre gestion des compteurs qui, bien que limitée par une borne, disons B pour l'exposé qui est fait ici, ne nécessite aucune allocation (exercices 1 et 2) ou alors une seule allocation (exercices 3 et suivants). Adaptez pour cela l'artifice de programmation introduit dans l'exercice 1.1 de la fiche de TP n°8 d'*Algorithmique 2*, avec l'idée supplémentaire qu'une fois franchie B , il n'y a plus à compter les occurrences d'un élément donné : il y en a simplement « beaucoup ».

Vous êtes libre dans le choix de B . Il est toutefois conseillé de fixer cette borne à quelques unités dans un premier temps, histoire de tester sa bonne gestion.

Exercice 1

Travail dans `hashtable_test/main.c` pour la suite.

Introduisez une macroconstante pour coder la borne B . Dans la foulée, déclarez le tableau associé aux compteurs en tant que variable globale. Adaptez le codage. Prévoyez pour la fonction `scptr_display` qu'un avertissement doit être produit lorsque qu'un compteur associé au mot dépasse la borne B .

Produisez l'exécutable. Testez.

Vous devriez voir deux types d'améliorations : un nombre d'allocations moindre (testez sous `valgrind`) ; une exécution sensiblement plus rapide sur de gros fichiers (testez sous `time`). Quantifiez l'amélioration du nombre d'allocations.

Exercice 2

Faites une copie du travail précédent ; nommez-la `main1.c`.

Travail dans `hashtable_test/main.c` pour la suite.

Améliorez le codage précédent : le tableau ne doit plus être une variable globale mais une variable locale à la fonction `main`.

Adaptez le codage. Il vous faudra, entre autres choses, recourir à `holdall_apply_context2` au lieu de `holdall_apply_context`.

Produisez l'exécutable. Testez.

Exercice 3

Faites une copie du travail précédent ; nommez-la `main2.c`.

Travail dans `hashtable_test/main.c` pour la suite.

Améliorez encore le codage précédent : c'est cette fois l'utilisateur qui fixe la borne B . Cette borne est lue sur la ligne de commande et la fonction `main` alloue dynamiquement le tableau associé.

Adaptez le codage. Comme argument à `holdall_apply_context2`, il vous faudra cette fois donner l'adresse d'un contexte qui sera une structure.

Produisez l'exécutable. Testez.

Exercice 4

Faites une copie du travail précédent ; nommez-la `main3.c`.

Travail dans `hashtable_test/main.c` pour la suite.

Améliorez encore le codage précédent. L'utilisateur doit dorénavant donner trois paramètres sur la ligne de commande. Le premier, comme précédemment, fixe la borne B . Le deuxième fixe une valeur de compteur : ne doivent être affichés que les mots dont le compteur est égal à cette valeur. Le troisième enfin fixe le nombre maximal de mots qui doivent être affichés.

Adaptez le codage. Il vous faudra compléter la structure du contexte.

Produisez l'exécutable. Testez.

Exercice 5

Affichez les mots dans l'ordre lexicographique (selon l'ordre induit par la fonction standard `strcmp`). Définissez à cet effet dans le fichier `holdall.c` la fonction `holdall_sort` déclarée dans le fichier en-tête `"holdall.h"`. Adaptez le source `holdall_ip.h` et le fichier `makefile` en conséquence. Testez.

Exemples.

```
$ ./test 10 5 100 < ../hashtable/hashtable.h
*** Warning: Word '--v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word '---v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word '--^---^---^---^---^---^---^---^...' possibly sliced
*** Warning: Word '---^---^---^---^---^---^---^---^...' possibly sliced
*** Warning: Word '-v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word '--v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word 'hashtable_fprint_stats(hashtabl...' possibly sliced
*** Warning: Word '-^---^---^---^---^---^---^---^...' possibly sliced
*** Warning: Word '--^---^---^---^---^---^---^---^...' possibly sliced
--- Info: Number of distinct words: 375
5----->*ht,
5----->Renvoie
5----->clé
5----->const
5----->hashtable
5----->ht
5----->macro
5----->pas
5----->qui
5----->si
5----->sont
5----->struct
5----->type
```

```

$ ./test 10 5 4 < ../hashtable/hashtable.h
*** Warning: Word '--v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word '---v---v---v---v---v---v---v---...' possibly sliced
*** Warning: Word '---^---^---^---^---^---^---^---...' possibly sliced
*** Warning: Word '---^---^---^---^---^---^---^---...' possibly sliced
*** Warning: Word '-v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word '--v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word 'hashtable_fprint_stats(hashtabl...' possibly sliced
*** Warning: Word '-^---^---^---^---^---^---^---...' possibly sliced
*** Warning: Word '--^---^---^---^---^---^---^---...' possibly sliced
--- Info: Number of distinct words: 375
5----->*ht,
5----->Renvoie
5----->clé
5----->const
$ ./test 4 5 4 < ../hashtable/hashtable.h
*** Warning: Word '--v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word '---v---v---v---v---v---v---v---...' possibly sliced
*** Warning: Word '---^---^---^---^---^---^---^---...' possibly sliced
*** Warning: Word '---^---^---^---^---^---^---^---...' possibly sliced
*** Warning: Word '-v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word '--v---v---v---v---v---v---v---v...' possibly sliced
*** Warning: Word 'hashtable_fprint_stats(hashtabl...' possibly sliced
*** Warning: Word '-^---^---^---^---^---^---^---...' possibly sliced
*** Warning: Word '--^---^---^---^---^---^---^---...' possibly sliced
--- Info: Number of distinct words: 375
*** Warning: Counter overflow for word '*ht,'
5----->*ht,
*** Warning: Counter overflow for word '- '
5----->-
*** Warning: Counter overflow for word '//'
5----->>//
*** Warning: Counter overflow for word ':'
5----->:

```

Algorithmique 3 : structures de données arborescentes

Fiche de TP n° 2

Implantation du TDA arbre binaire

Objectifs : travail sur un module implantant le TDA ABin ; génération de parties de code à l'aide de macrofonctions.

Prérequis : chapitre du cours d'*Algorithmique 2* sur les macrofonctions, chapitre du cours d'*Algorithmique 3* sur le TDA ABin.

Travail minimum : exercice 1.

Commencez par récupérer le dossier 2/, à trouver dans le dossier `algo3_src/tp/`.

Le dossier 2/ comprend deux sous-dossiers : `bt/`, contenant un module pour la spécification ABIN du TDA ABin dont la partie implantation est à compléter ; `bt_test/`, pour conduire des tests sur le module.

Placez-vous dans le sous-dossier `2/bt_test/`. Construisez l'exécutable. Exécutez-le. Dans la fenêtre active, vous devriez voir affichée la représentation formelle du peigne gauche de taille 4, ainsi que sa taille, sa hauteur et sa distance :

```
$ ./test
with t = bt_comb_left(4)
    bt_repr_formal(t) = ((((( ) ) ) ) ) )
    bt_size(t) = 4
    bt_height(t) = 4
    bt_distance(t) = 1
```

Prenez connaissance du contenu du fichier en-tête `"bt.h"`. Puis de celui du source `main.c` pour les tests. Observez que de nombreuses de ses lignes ont été mises sous voile (en commentaire donc) : soit elles ne peuvent être compilées car les fonctions qui y figurent ne sont pas définies ; soit leur exécution ferait planter le programme car le pointeur vers le contrôleur d'arbre binaire a pour valeur un pointeur nul. Prenez enfin connaissance du contenu du source `bt.c`.

Votre objectif premier : terminer le travail entamé. C'est précisément l'objet de l'exercice 1. Vous pourrez aussi aller un peu plus loin. C'est ce que propose l'exercice 2.

Exercice 1

Développez la partie implantation du module `bt`. Dévoilez conjointement les parties du programme qui peuvent l'être. Complétez conjointement le fichier `bt_ip.h`.

Précisions :

— `cbt__repr_graphic` : pour les nœuds, utilisez « 0 », pour les arbres vides, « | » ; l'indentation est précisée dans le source `bt.c` ; ne cherchez pas à dessiner les branches. Pour le peigne gauche de taille 4, après avoir dévoilé la ligne 44 du source `main.c`, vous devez obtenir :

```
$ ./test
with t = bt_comb_left(4)
    bt_repr_graphic(t) =
    |
0
    |
    0
        |
```

```

      0
      |
    0  |
      |

```

```

bt_repr_formal(t) = (((() ( ) ) ( ) ) ( ) )
bt_size(t) = 4
bt_height(t) = 4
bt_distance(t) = 1

```

— `cbt__comb_right`, `cbt__random`, `bt_comb_right` et `bt_random` : ces fonctions doivent être développées sur le même modèle que `cbt__comb_left` et `bt_comb_left` notamment en ce qui concerne la détection d'erreur ;

— `cbt__random` : pour générer un entier pseudo-aléatoire du type `size_t` strictement inférieur à la valeur du paramètre `n`, recourez à la formule

```
(size_t) ((double) rand() / (RAND_MAX + 1.0) * (double) n);
```

— il est souhaité que des fonctions de codes presque identiques soient générées à l'aide d'une macrofonction à l'instar de ce qui a déjà été entamé pour les trois fonctions de mesure `cbt__size`, `cbt__height` et `cbt__distance` ;

— supprimez les commentaires qui annoncent un travail à faire (les commentaires qui débutent par « `// ICI, PROCHAINEMENT, ...` ») dès que ce travail a été réalisé.

Exercice 2

En altérant la spécification, le prototype et la définition de la fonction `cbt__repr_graphic`, et donc, le cas échéant, son appel dans la fonction `bt_repr_graphic` :

1) Apportez quelques informations dans la représentation graphique en affichant le numéro de chacun des sous-arbres et le rang de chacun des nœuds. Un exemple parmi les quelques-uns obtenus après avoir dévoilé les lignes 64-74 :

```

with t = bt_random(4)
  bt_repr_graphic(t) =
    | no.=7
  0 no.=3
    | no.=6
0 no.=1
    | no.=11
    0 no.=5
      | no.=10
    0 no.=2
      | no.=4

  bt_repr_formal(t) = (((() ( ) ( )) ) ( ) ( )) )
  bt_repr_lukas(t) = aababbabb
  bt_repr_subtrees(t) = 11010000
  bt_size(t) = 4
  bt_height(t) = 3
  bt_distance(t) = 2
  bt_is_skinny(t) = false
  bt_is_comb_left(t) = false
  bt_is_comb_right(t) = false

```


2) Même chose, mais affichez cette fois le numéro au format binaire. Pour le seul affichage de l'arbre ci-dessus :

```
with t = bt_random(4)
    bt_repr_graphic(t) =
    | no.b=11
0 no.b=1
    | no.b=1011
    0 no.b=101
        | no.b=10101
        0 no.b=1010
            | no.b=10100
            0 no.b=10
                | no.b=100
```

Testez d'autres tailles en modifiant la valeur de la macroconstante **SIZE** définie dans **main.c**.

3) Améliorez la représentation graphique en ajoutant les branches. Cela peut par exemple donner :

```
with t = bt_random(4)
    bt_repr_graphic(t) =
    /---|
/---0
| \---|
0
|      /---|
| /---0
| | \---|
\---0
    \---|
```

Algorithmique 3 : structures de données arborescentes

Fiche de TP n° 3

Implantation du TDA arbre binaire de recherche (1)

Objectifs : travail sur un module implantant le TDA ABinR.

Prérequis : chapitre du cours sur le TDA ABinR ; fiche de TP n° 2.

Travail minimum : exercices 1 et 2.

Commencez par récupérer le dossier 3/, à trouver dans le dossier `algo3_src/tp/`.

Placez-vous dans le sous-dossier `3/bst_test/`. Construisez l'exécutable, `bstgame`. Lancez-le puis tapez « ? » après l'invite « > ». Vous devriez voir affiché dans la fenêtre active la liste des commandes auxquelles il est prévu que le jeu réagisse. Pour l'instant, le jeu ne réagit positivement qu'aux commandes « d », « h », « s », « V », « v », « ? » et « q ».

Prenez connaissance du contenu du fichier en-tête `"bst.h"` à trouver dans le sous-dossier `3/bst/`. Puis du source `main.c`. Observez qu'en dessous de la définition de la fonction principale `main` nombreuses de ses lignes ont été mises sous voile : elles ne peuvent être compilées car les fonctions préfixées par `bst_` qui y figurent ne sont pas définies. Prenez enfin connaissance du fichier `bst.c` à trouver dans le sous-dossier `3/bst/`.

Votre objectif premier : terminer le travail entamé. C'est précisément l'objet des exercices 1 à 3. Vous pourrez aussi aller un peu plus loin. C'est ce que propose l'exercice 4.

Exercice 1

1) Adaptez les diverses fonctions d'affichage développées à l'occasion de la fiche précédente : il faut cette fois afficher la valeur (de la racine) et la hauteur de chacun des sous-arbres non vides. Pour l'affichage de la valeur, vous devez utiliser la fonction `put` passée comme paramètre à la fonction `bst_repr_graphic`.

2) Développez conjointement la fonction de recherche `bst_search` et sa fonction auxiliaire, `cbst__search`, laquelle peut être récursive ou itérative. Conseil : utilisez la macrofonction `NEXT` qui, si son deuxième paramètre est, par exemple, le résultat non nul de la fonction `compar`, permet d'accéder au « bon suivant » du sous-arbre courant.

3) Développez conjointement la fonction d'ajout `bst_add_endofpath` ainsi que sa fonction récursive auxiliaire, `cbst__add_endofpath`. Même conseil.

4) Testez après avoir dévoilé puis remis en forme les lignes 175-201, 248-254 et 269 du source `main.c`. En cas de succès, retirez l'attribut `[[maybe_unused]]` situé devant le paramètre de la définition de `repr_graphic`. Complétez le fichier `bst_ip.h`.

Exercice 2

1) Développez conjointement la fonction de retrait `bst_remove_climbup_left` et ses fonctions auxiliaires `cbst__remove_max`, récursive, `cbst__remove_root` et `cbst__remove_climbup_left`, récursive. Conseil : utilisez la macrofonction `NEXT` dans la troisième.

2) Testez après avoir dévoilé puis remis en forme les lignes 207-228 du source `main.c`. Complétez le fichier `bst_ip.h`.

Exercice 3

1) Développez conjointement les fonctions `bst_number` et `bst_rank` et leurs fonctions auxiliaires récursives `cbst__number` et `cbst__rank`.

2) Testez après avoir dévoilé puis remis en forme les lignes 234-235 et 241-242 du source `main.c`. Complétez le fichier `bst_ip.h`.

Exercice 4

Les calculs des taille, hauteur et distance s'exécutent pour l'instant en temps linéaire en la taille de l'arbre. Ils peuvent en fait tous être réalisés en temps constant dès lors que chacun des sous-arbres non vides mémorise sa propre taille, hauteur et distance. Il est en revanche nécessaire de mettre à jour ces quantités dès qu'une modification, par ajout ou par retrait d'une valeur, est apportée à l'arbre. Observons que de telles mises à jour ne sont nécessaires que sur le chemin qui mène de la racine jusqu'au sous-arbre de plus haut niveau qui a été modifié.

Il est proposé ici de réaliser l'implantation en temps constant d'un seul des calculs des trois mesures : la taille.

1) Faites une copie du travail précédent. Nommez cette copie `bst_backup.c`¹. Travaillez pour la suite sur le source `bst.c`.

2) Modifiez la définition du type `cbst` en introduisant le champ `size` du type `size_t`.

3) Introduisez le raccourci

```
#define SIZE(p) ((p)->size)
```

au même endroit que les autres « *Raccourcis cbst* ».

4) Redéfinissez la fonction

```
static size_t cbst__size(const cbst *p);
```

de telle sorte qu'elle renvoie la taille du sous-arbre pointé par `p` cette fois en temps constant, que ce sous-arbre soit vide ou non (une ligne de code avec une expression alternative `?:`).

5) Définissez la fonction

```
static void cbst__update_size(cbst *p);
```

qui met à jour la taille du sous-arbre non vide pointé par `p` en temps constant (une ligne toujours).

6) Dans `cbst__add_endofpath`, initialisez la taille avec `SIZE` et mettez-la à jour lorsqu'il est nécessaire de le faire avec `cbst__update_size`.

7) Dans chacune des trois fonctions préfixée par `cbst__remove`, mettez à jour la taille lorsqu'il est nécessaire de le faire avec `cbst__update_size`.

8) Testez. Complétez le fichier `bst_ip.h`.

1. La copie pourra servir de point de départ pour le travail à réaliser à l'occasion d'une prochaine fiche de TP.

Algorithmique 3 : structures de données arborescentes

Fiche de TP n° 4

Autour des tas

Objectifs : travail sur un module implantant le TDA file de priorité à l'aide d'un tas et sur un autre implantant le tri par tas.

Prérequis : section du cours sur le TDA file de priorité, les tournois et les tas.

Travail minimum : exercices 1 et 2.

Commencez par récupérer le dossier 4/, à trouver dans le dossier `algo3_src/tp/`.

Le sous-dossier `heapsort/` est destiné aux exercices 2, 4, 5 et 6 ; le source `heapsort/main.c` est une adaptation du source du même nom du sous-dossier `algo2_src/tp/3/quicksort/`. Les trois autres sous-dossiers sont destinés aux exercices 1 et 3 ; le sous-dossier `holdall/` est une copie du sous-dossier de même nom du dossier `algo3_src/courstd/` ; le source `pqueue_test/main.c` est une adaptation du source du même nom du sous-dossier `algo3_src/courstd/hashtable_test/`.

Exercice 1

Prenez connaissance de l'en-tête `pqueue/pqueue.h`. Développez la partie implantation, `pqueue/pqueue.c`, comme spécifié dans ce dernier source : à l'aide d'un tas binaire dynamique. Utilisez les macroconstantes locales `CAPACITY_MIN` et `CAPACITY_MUL`. À la fonction de défilement `pqueue_dequeue`, donnez pour commencer — une alternative est proposée à l'exercice 3 — un codage classique : une descente comme dans le problème du « tournoi sauf à la racine » en procédant par échanges.

Testez (`valgrind`, `time`) sur de petits puis de gros textes.

Exercice 2

Développez le tri par tas, `heapsort`, dans le source `heapsort/heapsort.c`. Recourrez à la fonction locale `heapsort_down`, spécifiée et pré-déclarée dans ce source (à décommenter). Pour commencer — des alternatives sont proposées aux exercices 4 et 5 — donnez-lui un codage des plus classiques : une descente comme dans le problème du « tournoi sauf à la racine » en procédant par échanges ; définissez-la récursivement ; utilisez directement les indices dans les calculs d'adresses (« `base + indice * size` »).

Testez avec, par exemple, `-DxQSORT` à la place de `-DQSORT` dans le fichier `makefile`.

Exercice 3

Reprenez le codage de la fonction de défilement `pqueue_dequeue` de `pqueue/pqueue.c`. Cette fois la descente est complète, tout en bas du chemin des minimums avec glissements d'un cran vers le haut, sans effectuer la moindre comparaison avec la référence à placer, suivie de glissements d'un cran vers le bas des références sur le chemin des minimums, en remontant celui-ci, jusqu'à trouver la bonne place.

Pour rendre toujours possible l'accès à la version précédente, proposez une gestion avec une macroconstante définie à la compilation, `DEQUEUE`, dont les deux valeurs possibles sont `PART_SWAP`, associée à la version précédente, et `FULL_SHIFT`, associée à la présente version.

Testez.

Exercice 4

Reprenez le codage de la fonction `heapsort_down` de `heapsort/heapsort.c` : dérécursifiez ; remplacez les indices par les décalages associés dans les calculs d'adresses (« `base + décalage` »).

Pour rendre toujours possible l'accès à la version précédente, proposez une gestion avec une macroconstante définie à la compilation, `DOWN`, dont les deux valeurs possibles sont `PART_SWAP_INDEX`, associée à la version précédente, et `PART_SWAP_OFFSET`, associée à la présente version.

Testez.

Exercice 5

Pour la partie tri à proprement parler, développez puis utilisez l'autre fonction locale spécifiée et prédéclarée dans le source `heapsort/heapsort.c` : `heapsort_down_from_0`. Implantez-la avec décalages, comme précédemment, et avec une descente complète tout en bas du chemin des minimums avec glissements d'un cran vers le haut, sans effectuer la moindre comparaison avec la valeur à placer, suivie de glissements d'un cran vers le bas des valeurs sur le chemin des minimums, en remontant celui-ci, jusqu'à trouver la bonne place. Afin que les glissements vers le haut puissent avoir lieu, autorisez-vous la mémorisation du composant d'indice 0 dans une zone mémoire de taille `size` automatique (dans la pile donc). Pour les copies, utilisez la fonction standard `memcpy`.

Ajoutez `FULL_SHIFT_OFFSET` comme valeur possible pour `DOWN`.

Testez.

Exercice 6

Effectuez des tests sous `time` en comparant avec `qsort`. Pour ce faire, dans le fichier `makefile`, désactivez l'option `-DECHO`, en donnant du `-DxECHO` par exemple, jouez entre les options `-DQSORT` / `-DxQSORT`, fixez la longueur, `LENGTH`, avec « `-DLENGTH=...` », et le nombre de tests², `TRIALS`, avec « `-DTRIALS=...` ».

2. Pour le nombre de tests, vous pouvez aussi vous laisser faire sans fixer `LENGTH` par vous-même. Si toutefois le temps d'attente vous paraît trop long, modifier la valeur de la macroconstante `TRIALS_RATIO_DEFAULT` définie dans `main.c` en en prenant une 2 à 10 fois plus faible. Si à l'inverse le temps des tests vous paraît trop court, modifiez-la pour une valeur 2 à 10 fois plus élevée.

Algorithmique 3 : structures de données arborescentes

Fiche de TP n° 5

Implantation du TDA arbre binaire de recherche (2)

Objectifs : travail sur un module implantant le TDA ABinR.

Prérequis : chapitres du cours sur le TDA ABinR et les arbres binaires équilibrés ; fiche de TP n° 3.

Travail minimum : exercices 1 à 4.

Faites une copie du dossier consacré à la fiche n° 3. Nommez ce nouveau dossier selon votre nomenclature : c'est dans ce dernier que vous allez travailler. Renommez `avl/` son sous-dossier `bst/`. Modifiez la ligne 4 du fichier `makefile` et la valeur de la variable `bst_dir` du fichier `bst_test/makefile` en conséquence.

Exercice 1

Calcul de la hauteur en temps constant.

Pour faire en sorte que la hauteur de chaque sous-arbre non vide situé sur le chemin issu de la racine emprunté lors des opérations d'ajout et de retrait soit mise à jour en temps constant, il est nécessaire de mémoriser dans chaque cellule de type `cbst` la hauteur du sous-arbre non vide associé.

1) Dans `avl/bst.c`, en descendant sur le code :

a) modifiez la définition du type `cbst` en introduisant, immédiatement après le champ `ref`, le champ `height` du type `int` ;

b) introduisez entre `REF` et `NEXT` le raccourci

```
#define HEIGHT(p) ((p)->height)
```

c) remplacez toutes les occurrences de `size_t` par `int` dans la fonction locale de calcul du maximum (aussi la nouvelle fonction se nomme-t-elle `max__int`) ;

d) remplacez la définition de la fonction récursive `cbst__height` générée à l'aide de la macro-fonction `DEFUN_CBST_MEASURE` par celle de la fonction de prototype

```
static int cbst__height(const cbst *p);
```

qui renvoie la hauteur du sous-arbre associé à `p` en temps constant, que ce sous-arbre soit vide ou non, sous l'hypothèse que, si le sous-arbre n'est pas vide, le champ `height` de `*p` est à jour (une ligne de code avec une expression alternative `?:`) ;

e) définissez immédiatement en dessous la fonction de prototype

```
static void cbst__update_height(cbst *p);
```

qui met à jour la hauteur du sous-arbre non vide associé à `p` en temps constant, sous l'hypothèse que le champ `height` de ses propres sous-arbres non vides est à jour (une ligne en utilisant `max__int` et `cbst__height`) ;

f) dans `cbst__add_endofpath`, ajoutez, selon les cas, l'initialisation du champ `height` et sa mise à jour par `cbst__update_height` ;

g) dans `cbst__remove_max`, `cbst__remove_root` et `cbst__remove_climbup_left`, ajoutez la mise à jour du champ `height` par `cbst__update_height` ;

h) adaptez la définition de la fonction `cbst__repr_graphic`, y compris le type de son éventuel paramètre associé au niveau ;

i) adaptez la définition de la fonction `bst_height`.

2) Produisez l'exécutable. Testez : ajouts, retraits. Ne passez à la suite que si les hauteurs affichées sont toutes cohérentes.

MÀJ 31-03
Il y a une incohérence dans le sujet du TP concernant le nom de la macro-fonction. En effet, dans le fichier C, la macro est nommée `DEFUN_CBST__MEASURE`, tandis que dans le support de TP, elle est indiquée comme `DEFUN_CBST_MEASURE`.
(Hamza)

Exercice 2

Introduction de l'équilibre, calcul comme affichage.

1) Dans `avl/bst.c` :

a) immédiatement après la définition de `cbst__update_height`, définissez la fonction de prototype

```
static int cbst__balance(const cbst *p);
```

qui renvoie l'équilibre du sous-arbre associé à `p` en temps constant, que ce sous-arbre soit vide ou non, sous l'hypothèse que le champ `height` de ses propres sous-arbres non vides est à jour (une ligne de code avec une expression alternative `?:`);

b) adaptez l'affichage en ajoutant l'équilibre de chacun des sous-arbres non vides.

2) Dans `bst_test/main.c`, modifiez de conserve la fin du message d'aide de l'opération d'affichage, de commande `'g'`, en passant de

```
"_Each_node_value_is_followed_by_the_height_of_the_associated_subtree",
```

à

```
"_Each_node_value_is_followed_by_the_height_and_the_balance_of_the"
"_associated_subtree",
```

3) Produisez l'exécutable. Testez. Ne passez à la suite que si les balances affichées sont toutes cohérentes.

Exercice 3

Passage aux AVL.

1) Dans `avl/bst.c` :

a) Définissez immédiatement en dessous de la fonction `cbst__balance` les procédures de prototypes

```
static void cbst__rotation_left(cbst **pp);
static void cbst__rotation_right(cbst **pp);
```

qui, supposant que le sous-arbre associé à `*pp` supporte la rotation en question, effectuent ladite rotation et mettent à jour les hauteurs des arbres impactés *via* la fonction `cbst__update_height` (six lignes de code pour chacune des deux procédures);

b) sous la même hypothèse, déduisez-en l'expression des deux procédures de rotations composées (deux lignes de code pour chacune des deux procédures)

```
static void cbst__rotation_left_right(cbst **pp);
static void cbst__rotation_right_left(cbst **pp);
```

c) définissez ensuite la fonction de rééquilibrage éventuel

```
static int cbst__balancing(cbst **pp);
```

laquelle commence par mettre à jour la hauteur du sous-arbre non vide pointé par `*pp` *via* la fonction `cbst__update_height`, puis renvoie zéro si le sous-arbre est équilibré, et, dans le cas contraire, effectue la rotation nécessaire puis renvoie une valeur non nulle³ (moins de vingt lignes de code, sans difficulté aucune, suffisent);

d) dans vos fonctions d'ajout (`cbst__add_endofpath` donc) et de retrait (les trois fonctions préfixées par `cbst__remove`), remplacez les occurrences des formes `cbst__update_height(ptr)` par `cbst__balancing(référence-de-ptr)`.

2) Produisez l'exécutable. Testez.

3. La valeur de retour pourra être exploitée dans la version itérative de l'ajout à l'exercice 5.

Exercice 4*Finitions.*

- 1) Accordez au calcul local de la distance le même type que la hauteur : **int**.
- 2) Dans `bst_test/main.c`, modifiez les valeurs de macroconstantes **BST** et **BST_L** définies lignes 13 et 14 en **"AVL"** et **"AVL_binary_search_tree"**. Modifier `bst_test/makefile` afin qu'il nomme l'exécutable `avlgame`.
- 3) Faites en sorte que le source `avl/bst.c` considère la possible définition préalable d'une macroconstante, de telle sorte que si celle-ci est définie, le nom de toute rotation effectuée est affiché sur la sortie.

Exemple. Avec l'arbre de Fibonacci valué de rang (et donc de hauteur) 6 :

```
$ echo "+13+8+18+5+11+16+20+3+7+10+12+15+17+19+2+4+6+9+14+1g-20gq" | ./avlgame
--- add '13' to the AVL
```

```
...
```

```
--- add '1' to the AVL
```

```

      |
    20 h=2 b=1
      |
    19 h=1 b=0
      |
    18 h=4 b=1
      |
    17 h=1 b=0
      |
    16 h=3 b=1
      |
    15 h=2 b=1
      |
    14 h=1 b=0
      |
    13 h=6 b=1
      |
    12 h=1 b=0
      |
    11 h=3 b=1
      |
    10 h=2 b=1
      |
     9 h=1 b=0
      |
    8 h=5 b=1
      |
     7 h=2 b=1
      |
     6 h=1 b=0
      |
    5 h=4 b=1
      |
```

```

      4 h=1 b=0
      |
    3 h=3 b=1
      |
    2 h=2 b=1
      |
    1 h=1 b=0
      |
--- remove '20' to the AVL
right
right
      |
    19 h=1 b=0
      |
    18 h=2 b=0
      |
    17 h=1 b=0
      |
    16 h=3 b=0
      |
    15 h=2 b=1
      |
    14 h=1 b=0
      |
    13 h=4 b=0
      |
    12 h=1 b=0
      |
    11 h=3 b=1
      |
    10 h=2 b=1
      |
    9 h=1 b=0
      |
    8 h=5 b=0
      |
    7 h=2 b=1
      |
    6 h=1 b=0
      |
    5 h=4 b=1
      |
    4 h=1 b=0
      |
    3 h=3 b=1
      |
    2 h=2 b=1
      |
    1 h=1 b=0

```

--- May the AVL-force be with you!

4) Mettez enfin à jour `avl/bst_ip.h`.

Exercice 5

Dérécursifications (1).

La hauteur des arbres binaires de recherche est désormais raisonnable. Et l'estimer majorée par 48 par exemple autorise la gestion d'arbres de tailles inférieures à $2,8 \cdot 10^{10}$.

Profitez-en pour donner aux opérations de recherche, d'ajout puis de retrait `bst_...` des corps uniquement itératifs, sans plus d'appels aux fonctions récursives `cbst_...`.

1) Dans votre dossier consacré à l'actuelle fiche de TP, faites une copie du sous-dossier `avl/`. Nommez-la `avl_stacked/` : c'est le source `avl_stacked/bst.c` que vous allez transformer dans la suite. Modifiez la variable `bst_dir` du fichier `bst_test/makefile` en conséquence.

2) Échauffez-vous en dérécursifiant `cbst__search` : il s'agit là d'une application immédiate de résultats d'*Algorithmique 2*. Testez.

3) Dérécursifiez `cbst__add_endofpath` en utilisant un tableau de composants du type `cbst **` dont l'objet est d'empiler les adresses des pointeurs associés à la descente dans l'arbre. Tâchez d'utiliser au mieux les résultats du cours en réinitialisant la pile à chaque sous-arbre de hauteur non nulle rencontré lors de la descente et en stoppant la remontée dès qu'une rotation a été effectuée. Testez.

4) Fondez les fonctions pour le retrait, `cbst_remove_climbup_left`, `cbst_remove_root` et `cbst_remove_max`, en dérécursifiant la première et la troisième comme précédemment. Testez.

Exercise 6

Dérécursifications (2).

1) Attaquez-vous à `cbst__number` et `cbst__rank` (immédiat).

2) Enchaînez avec `cbst_dispose`, à l'aide d'une pile de `cbst **`, comme dans l'exercice précédent (un peu moins facile).

3) Puis avec `cbst__size` (même ordre de difficulté). Vous laisserez cette fonction dans la partie `cbst` : elle est utilisée par `cbst__rank`.

4) Poursuivez avec `cbst__repr_graphic`. Il vous faudra évidemment cette fois stocker dans la pile d'autres informations que des simples adresses de pointeurs (toujours moins facile).

5) Terminez avec le calcul de la distance, `cbst__distance`. Il s'agit cette fois-ci de faire face à un parcours nécessairement postfixe et à la mémorisation des valeurs de retour (difficile).

6) Mettez enfin à jour `avl_stacked/bst_ip.h`.

MÀJ 17-03
de⊕s⊕
arbres
(AL)

Algorithmique 3 : structures de données arborescentes

Fiche de TP n° 6

Compression et décompression à la Huffman

Objectifs : implantations d'arbres binaires étendus pondérés en tableau.

Prérequis : chapitre du cours sur les arbres de Huffman.

Travail minimum : exercices 1 et 2.

Tout d'abord en clair, en ne produisant, exercice 1, ou ne traitant, exercice 2, que des caractères '0' et '1'. Puis en passant aux bits en paquets, portés par des **unsigned char** : exercices 3 et 4.

Faites nécessairement dépendre votre codage des macroconstantes standard **UCHAR_MAX** et **CHAR_BIT**. Rappelons que leurs valeurs sont liées : $\text{lb}(\text{UCHAR_MAX} + 1) = \text{CHAR_BIT}$.

Pour les compressions, exercices 1 et 3, la structure suivante suffit pour les feuilles et les nœuds :

```
typedef struct {
    long int wgt;
    int prev;
    union {
        int next[2];
        int val;
    };
} hitem;
```

Et pour les décompressions, exercices 2 et 4 :

```
typedef struct {
    enum {
        NODE,
        LEAF,
        XLEAF
    } mark;
    union {
        int next[2];
        int val;
    };
} hitem;
```

la dernière des marques étant réservée à la feuille associée au marqueur de fin du texte.

Exercice 1

Confectionnez un source qui permet de réaliser ce que promet son aide :

```
$ ./hcompress01 --help
Usage: ./hcompress01 SRC [DEST]
```

```
Simulate Huffman compression of input file SRC by producing digits "0" and "1".
The output is written to DEST file if specified, to standard output otherwise.
```

Testez sur de petits puis de gros fichiers. Notez les tailles des sources et des productions; calculez les taux de compressions après avoir divisé par 8 les tailles des productions.

Exercice 2

Confectionnez un source qui permet de réaliser ce que promet son aide :

```
$ ./unhcompress01 --help
```

```
Usage: ./unhcompress01 SRC [DEST]
```

Simulate Huffman decompression of input file SRC. SRC file shall have been produced by an Huffman compression simulator and shall therefore only be composed of digits "0" and "1". The output is written to DEST file if specified, to standard output otherwise.

Testez à la suite de la commande précédente; comparez (diff) le résultat de la présente commande avec la donnée de la commande précédente.

Exercice 3

Confectionnez un source qui permet de réaliser ce que promet son aide :

```
$ ./hcompress --help
```

```
Usage: ./hcompress SRC [DEST]
```

Make Huffman compression of input file SRC. The output is written to DEST file if specified, to standard output otherwise.

When SRC is "-", read standard input. When DEST is "-", write to standard output.

N'abordez la possibilité de considérer l'opérateur spécial « - » que dans un deuxième temps.

Exercice 4

Confectionnez un source qui permet de réaliser ce que promet son aide :

```
$ ./unhcompress --help
```

```
Usage: ./unhcompress SRC [DEST]
```

Make Huffman decompression of input file SRC. SRC file shall have been produced by an Huffman compression maker. The output is written to DEST file if specified, to standard output otherwise.

When SRC is "-", read standard input. When DEST is "-", write to standard output.

Même conseil.